

Design patterns, the big picture, Part 1: Design pattern history and classification

Understanding the concept and use of design patterns in software development

By Jeff Friesen, JavaWorld.com, 11/21/12

Design patterns are a cornerstone of modern software development. Using them can improve the maintainability of your Java applications and help you accomplish other tasks more effectively. Jeff Friesen's three-part introduction to design patterns takes a wide-angle view, from the GoF patterns to design patterns used in interaction design, team organization, and more. Part 1 introduces the concept and use of design patterns, offers some design pattern history and trivia, and ends with a listing of software design patterns by classification. Included with this article is a [complete index](#) of design pattern tutorials on JavaWorld.

Numerous strategies have been devised to simplify and reduce the costs of designing software, especially in the area of maintenance. Learning how to identify and work with reusable software components (occasionally referred to as *software integrated circuits*) is one strategy. Using design patterns is another.

This article launches a three-part series on design patterns. In this part I'll introduce the conceptual framework of design patterns and walk through a demonstration of evaluating a design pattern for a particular use case. I'll also discuss the history of design patterns, including some interesting trivia. Finally, I'll classify and summarize the most-used software design patterns that have been discovered and documented through the past couple of decades.

- [What is a design pattern?](#)
- [Anti-patterns](#)
- [Design pattern history](#)
- [Software design patterns classified](#)

What is a design pattern?

Designing reusable object-oriented software that models an existing system is genuinely challenging. A software developer must factor the system's entities into classes whose public interfaces aren't overly complex, establish relationships among classes, expose inheritance hierarchies, and more. Because most software remains in use long after it was written, software developers also need to address current application requirements while keeping their code and infrastructure flexible enough to meet future needs.

Experienced object-oriented developers have discovered that software design patterns facilitate the coding of stable and robust software systems. Reusing these design patterns rather than constantly developing new solutions from scratch is efficient, and it reduces some of the risk of error. Every design pattern identifies a recurring design problem in a specific application context, then offers a generalized, reusable solution that is applicable to different application scenarios.

"A *design pattern* describes the classes and interacting objects used to solve a general design problem in a specific context."

Some developers define a *design pattern* as a class-encoded entity (such as a linked list or bit vector), whereas others say that a design pattern is in the entire application or subsystem. My view is that a *design*

pattern describes the classes and interacting objects used to solve a general design problem in a specific context. More formally, a design pattern is specified as a description that consists of four basic elements:

1. A *name* that describes the design pattern and gives us a vocabulary for discussing it
2. A *problem* that identifies the design problem needing to be solved along with the context in which the problem occurs
3. A *solution* to the problem, which (in a software design pattern context) should identify the classes and objects that contribute to the design along with their relationships and other factors
4. An explanation of the *consequences* of using the design pattern

In order to identify the appropriate design pattern to use, you must first clearly identify the problem that you're trying to solve; that's where the *problem* element of the design pattern description is helpful. Choosing one design pattern over another also usually involves trade-offs that can impact an application's or system's flexibility and future maintenance. That's why it's important to understand the *consequences* of using a given design pattern before you begin implementing it.

Evaluating a design pattern

Consider the task of designing a complex user interface using buttons, textfields, and other non-container components. The [Composite](#) design pattern regards containers as components, which lets us nest containers and their components (containers and non-containers) within other containers, and do so recursively. If we chose not to use the Composite pattern we would have to create many specialized non-container components (a single component combining a password textfield and a login button, for example), which is harder to achieve.

Having thought this through, we understand the problem we're trying to solve and the solution offered by the Composite pattern. But what are the consequences of using this pattern?

Using Composite means that your class hierarchies will mix container and non-container components. Simpler clients will treat container and non-container components uniformly. And it will be easier to introduce new kinds of components into the UI. Composite can also lead to *overly generalized* designs, making it harder to restrict the kinds of components that can be added to a container. Since you won't be able to rely on the compiler to enforce type constraints, you will have to use runtime type checks.

What's wrong with runtime type checks?

Runtime type checks involve *if* statements and the `instanceof` operator, which leads to brittle code. If you forget to update a runtime type check as your application requirements evolve, you could subsequently introduce bugs.

It is also possible to choose an appropriate design pattern and use it incorrectly. The *Double-Checked locking* pattern is a classic example. Double-checked locking reduces lock acquisition overhead by first testing a locking criterion without actually acquiring the lock, and then only acquiring the lock if the check indicates that locking is required. While it looked good on paper, Double-checked locking in JDK 1.4 had some hidden complexities. When JDK 5 extended the semantics of the `volatile` keyword, developers were finally able to reap the benefits of the Double-checked locking pattern.

More about Double-checked locking

See "[Double-checked locking: Clever, but broken](#)" and "[Can double-checked locking be fixed?](#)" (Brian Goetz, JavaWorld) to learn more about why this pattern didn't work in JDK 1.4 and earlier. For more about

specifying DCL in JDK 5 and later, see "[The 'Double-Checked Locking is Broken' Declaration](#)" (University of Maryland Department of Computer Science, David Bacon, et al.).

Anti-patterns

When a design pattern is commonly used but is ineffective and/or counterproductive, the design pattern is known as an *anti-pattern*. One might argue that Double-checked locking as used in JDK 1.4 and earlier was an anti-pattern. I would say that it was merely a bad idea in that context. For a bad idea to evolve into an anti-pattern, the following conditions must be met (see [Resources](#)).

- A repeated pattern of action, process, or structure that initially appears to be beneficial, but ultimately produces more bad consequences than beneficial results.
- An alternative solution exists that is clearly documented, proven in practice, and repeatable.

While Double-checked locking in JDK 1.4 did meet the first requirement of an anti-pattern, it did not meet the second: although you could use `synchronized` to solve the problem of lazy initialization in a multithreaded environment, doing so would have defeated the reason for using Double-checked locking in the first place.

Deadlock anti-patterns

Recognizing anti-patterns is a prerequisite to avoiding them. Read Obi Ezechukwu's three-part series for an introduction to three anti-patterns famous for causing deadlock:

- [No arbitration](#)
- [Worker aggregation](#)
- [Incremental locking](#)

Design pattern history

Design patterns date back to the late 1970s with the publication of *A Pattern Language: Towns, Buildings, Construction* by architect Christopher Alexander and a few others. This book introduced design patterns in an architectural context, presenting 253 patterns that collectively formed what the authors called a *pattern language*.

The irony of design patterns

Although design patterns used for software design trace their beginning to *A Pattern Language*, this architectural work was influenced by the then-emerging language to describe computer programming and design.

The concept of a pattern language subsequently emerged in Donald Norman's and Stephen Draper's *User Centered System Design*, which was published in 1986. This book suggested the application of pattern languages to *interaction design*, which is the practice of designing interactive digital products, environments, systems, and services for human use.

Meanwhile, Kent Beck and Ward Cunningham had begun to study patterns and their applicability to software design. In 1987, they used a series of design pattern to assist Tektronix's Semiconductor Test Systems Group,

which was having trouble finishing a design project. Beck and Cunningham followed Alexander's advice for user-centered design (letting representatives of the project's users determine the design outcome) while also providing them with some design patterns to make the job easier.

Erich Gamma also realized the importance of recurring design patterns while working on his PhD thesis. He believed that design patterns could facilitate the task of writing reusable object-oriented software, and pondered how to document and communicate them effectively. Prior to the 1991 European Conference on Object-Oriented Programming, Gamma and Richard Helm started to catalog patterns.

At an OOPSLA workshop held in 1991, Gamma and Helm were joined by Ralph Johnson and John Vlissides. This *Gang of Four* (GoF), as they subsequently were known, went on to write the popular *Design Patterns: Elements of Reusable Object-Oriented Software*, which documents 23 design patterns in three categories.

The modern evolution of design patterns

Design patterns have continued to evolve since the original GoF book, especially as software developers have confronted new challenges related to changing hardware and application requirements.

In 1994, a U.S.-based non-profit organization known as the [Hillside Group](#) inaugurated *Pattern Languages of Programs*, a group of annual conferences whose aim is to develop and refine the art of software design patterns. These ongoing conferences have yielded many examples of domain-specific design patterns. For example, design patterns in a concurrency context.

Christopher Alexander at OOPSLA

OOPSLA 96's keynote address was delivered by the architect Christopher Alexander. Alexander reflected on his work and on how the object-oriented programming community had hit and missed the mark in adopting and adapting his ideas about pattern languages to software. You can read Alexander's address in full: "[The Origins of Pattern Theory: the Future of the Theory, And The Generation of a Living World.](#)"

In 1998 Mark Grand released *Patterns in Java*. This book included design patterns not found in the GoF book, including concurrency patterns. Grand also used the Unified Modeling Language (UML) to describe design patterns and their solutions. The book's examples were expressed and described in the Java language.

Software design patterns by classification

Modern software design patterns are broadly classified into four categories based on their use: creational, structural, behavioral, and concurrency. I'll discuss each category and then list and describe some of the prominent patterns for each one.

Other types of design patterns

If you're thinking that there are more types of patterns, you are right. A later article in this series will discuss additional design pattern types: Interaction, architectural, organizational, and communication/presentation patterns.

Creational patterns

A *creational pattern* abstracts the process of instantiation, separating how objects are created, composed, and represented from the code that relies on them. *Class creational patterns* use inheritance to vary the classes

that are instantiated, and *object creational patterns* delegate instantiation to other objects.

- **[Abstract factory](#)**: This pattern provides an interface to encapsulate a group of individual factories that have a common theme without specifying their concrete classes.
- **[Builder](#)**: Separates the construction of a complex object from its representation, enabling the same construction process to create various representations. Abstracting the steps of object construction allows different implementations of the steps to construct different representations of the objects.
- **[Factory method](#)**: Defines an interface for creating an object, but lets subclasses decide which class to instantiate. This pattern lets a class defer instantiation to subclasses. [Dependency injection](#) is a related pattern. (See [Resources](#).)
- **[Lazy initialization](#)**: This pattern gives us a way to delay object creation, database lookup, or another expensive process until the first time the result is needed.
- **[Multiton](#)**: Expands on the singleton concept to manage a map of named class instances as key-value pairs, and provides a global point of access to them.
- **[Object pool](#)**: Keep a set of initialized objects ready to use, rather than be allocated and destroyed on demand. The intent is to avoid expensive resource acquisition and reclamation by recycling objects that are no longer in use.
- **[Prototype](#)**: Specifies the kinds of objects to create using a prototypical instance, then create new objects by copying this prototype. The prototypical instance is cloned to generate new objects.
- **[Resource acquisition is initialization](#)**: This pattern ensures that resources are automatically and properly initialized and reclaimed by tying them to the lifespan of suitable objects. Resources are acquired during object initialization, when there is no chance of them being used before they are available, and released with the destruction of the same objects, which is guaranteed to take place even in the case of errors.
- **[Singleton](#)**: Ensures that a class has only one instance and provides a global point of access to this instance.

Structural patterns

A *structural pattern* teaches us how to compose classes and objects to form larger structures. A *structural class pattern* relies on inheritance to compose a resulting interface or implementation (for example, multiple inheritance mixes two or more classes into one class). A *structural object pattern* composes various objects to obtain new functionality; the [Composite pattern](#) is one example of this approach.

- **[Adapter](#)**: Converts a class's interface into another interface that clients expect. An adapter lets classes work together that otherwise couldn't due to incompatible interfaces. It does so by providing its interface to clients while using the original class interfaces internally. Adapter is also known as the Wrapper pattern.
- **[Bridge](#)**: Decouples an abstraction from its implementation, which lets the two vary independently. The *bridge* uses encapsulation, aggregation, and can also use inheritance to separate responsibilities into different classes.
- **[Composite](#)**: Composes objects into tree structures that represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
- **[Decorator](#)**: Dynamically attaches additional responsibilities to an object while maintaining the same interface. Decorators provide a flexible alternative to subclassing for extending functionality.
- **[Facade](#)**: Provides a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use. Facades can wrap poorly-designed APIs in a single well-defined API.
- **[Flyweight](#)**: Uses sharing to support large numbers of similar objects efficiently. A *flyweight* is an object that minimizes memory use by sharing as much data as possible with other similar objects. It offers a way to use objects in large numbers when a simple repeated representation would result in an

unacceptable amount of allocated memory.

- **Front controller**: Provides a centralized entry point for handling requests. This pattern relates to the design of Web applications.
- **Module**: Implements the concept of software modules, defined by modular programming, in a programming language that does not support or only partly supports modules.
- **Proxy**: Provides a surrogate or placeholder object for another object. This *proxy* controls access to the other object.

Behavioral patterns

Behavioral patterns focus on algorithms and the assignment of responsibilities between objects. They address object or class patterns as well as the communication patterns between them. A *behavioral class pattern* uses inheritance to distribute behavior among classes. In contrast, a *behavioral object pattern* uses object composition.

- **Blackboard**: This is a generalization of the Observer pattern that supports multiple readers and writers. The Blackboard pattern communicates information systemwide.
- **Chain of responsibility**: Avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. This pattern chains together receiving objects and passes a request along the chain until an object handles the request.
- **Command**: Uses an object to encapsulate all information needed to call a method at a later time. This information includes the method name, the object that owns the method, and values for the method parameters. A client instantiates the command object and provides the information required to call the method. The invoker decides when the method should be called. Finally, the receiver is an instance of the class that contains the method's code.
- **Interpreter**: Given a language, defines a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.
- **Iterator**: Enables developers to access the elements of an aggregate object sequentially without exposing its underlying representation.
- **Mediator**: Defines an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.
- **Memento**: Without violating encapsulation, captures and externalizes an object's internal state so that the object can be subsequently restored to this state.
- **Null Object**: Avoids null references by providing a default object.
- **Observer**: Defines a one-to-many dependency between objects where a state change in one object results in all of its dependents being notified and updated automatically. Observer is also known as the Publish-subscribe pattern.
- **Servant**: Defines functionality for a group of classes without defining that functionality in each of those classes. A *servant* is a class whose instance (or the class itself) provides methods that perform a desired service, while objects for which (or with whom) the servant does something are passed to servant methods as parameters.
- **Specification**: Recombines business rules by chaining the business rules together using Boolean logic.
- **State**: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.
- **Strategy**: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Template method**: Defines the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.
- **Visitor**: Represents an operation to be performed on the elements of an object structure. Visitor lets

you define a new operation without changing the classes of the elements on which it operates.

Concurrency patterns

Finally, a *concurrency pattern* addresses some aspect of multithreaded programming. One well-known pattern in this category is *Producer-consumer*, in which a producer thread stores an item in a shared buffer and a consumer thread retrieves this item. The producer thread must not store another item in this buffer until the previous item has been consumed, and the consumer thread must not consume a non-existent item.

- **Active object:** Decouples method execution from method invocation for objects in which each object resides in its own thread of control. The goal is to introduce concurrency, by using asynchronous method invocation and a scheduler for handling requests
- **Balking:** Only executes an action on an object when the object is in a particular state. For example, an object reads a file and provides methods to access file content. When the file is not open and an attempt is made to call a method to access content, the object "balks" at the request.
- **Binding properties:** Combines multiple observers to force properties in different objects to be synchronized or coordinated in some way.
- **Double-checked locking:** Reduces the overhead of acquiring a lock by first testing the locking criterion (the "lock hint") without actually acquiring the lock. The lock is acquired only when the locking criterion check indicates that locking is required.
- **Event-based asynchronous:** A concurrency pattern for the asynchronous invocation of an object's potentially long-running methods.
- **Guarded suspension:** Manages operations that require both a lock to be acquired and a precondition to be satisfied before the operation can be executed.
- **Lock:** A mechanism to temporarily make some aspect of an object unmodifiable or to suppress unneeded update notifications.
- **Messaging design pattern (MDP):** Allows the interchange of information (that is, messages) between components and applications.
- **Monitor object:** An object whose methods are subject to mutual exclusion, thus preventing multiple objects from erroneously trying to use it at the same time.
- **Reactor:** An event-handling pattern for handling service requests delivered concurrently to a service handler by one or more inputs. The service handler then demultiplexes the incoming requests and dispatches them synchronously to the associated request handlers.
- **Read-write lock:** A lock that allows concurrent read access to an object, but requires exclusive access for write operations.
- **Scheduler:** A concurrency pattern used to explicitly control when threads may execute single-threaded code; for example, multiple threads wanting to write data to the same file.
- **Thread pool:** A concurrency pattern in which a number of threads are created to perform various tasks, which are usually organized in a queue. Typically, there are many more tasks than threads. Thread Pool can be considered a special case of the Object Pool creational pattern.
- **Thread-specific storage:** A concurrency pattern in which static or global memory is localized to a thread. Each thread has its own copy of this memory.

The previous lists describe many commonly used (and some less commonly used) software design patterns and are not exhaustive. See [Resources](#) for further reading.

Conclusion to Part 1

Design Patterns: Elements of Reusable Object-Oriented Software significantly impacted the field of software design patterns. In Part 2, I'll present a helpful way to learn the foundational design patterns

presented by the Gang of Four authors, revisit the book's Visitor pattern from a Java technology perspective, and discuss some possible downsides of the GoF patterns and of using software design patterns generally. In the meantime, be sure to check the [Resources](#) section for a listing of design pattern tutorials on JavaWorld and elsewhere.

About the author

Jeff Friesen is a freelance tutor and software developer with an emphasis on Java and Android. In addition to writing Java and Android books for [Apress](#), Jeff has written numerous articles on Java and other technologies for JavaWorld, [informIT](#), [Java.net](#), and [DevSource](#). Jeff can be contacted via his website at [TutorTutor.ca](#).